



Enterprise Fraud Detection Platform

Detecting Financial Fraud In Real Time

Israel De La Mothe - Kieron Seepersad - Dylan Sinkia - Tyrese Des Vignes



The Problem

Detecting whether or not a transaction may be fraudulent or not in real time..

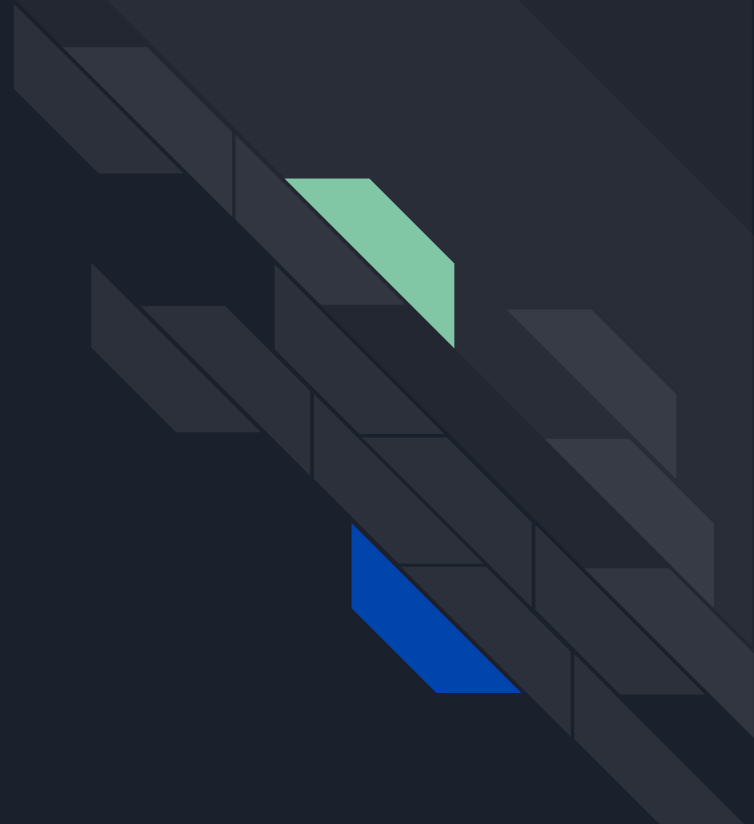
- Fraudsters now operate in coordinated rings and conduct account takeovers
- Traditional rule-based systems miss emerging, never before seen fraud patterns
- False negatives (missed fraud) cause direct financial loss and erode customer trust

ISSUES

- Extreme class imbalance
- High-velocity data
- Evolving patterns

Part 1

The Data





Data

PaySim mobile money transaction simulation from Kaggle

- Size: 6,362,620 transactions
- Time: 30 day simulated activity
- Rate: 0.13% fraud

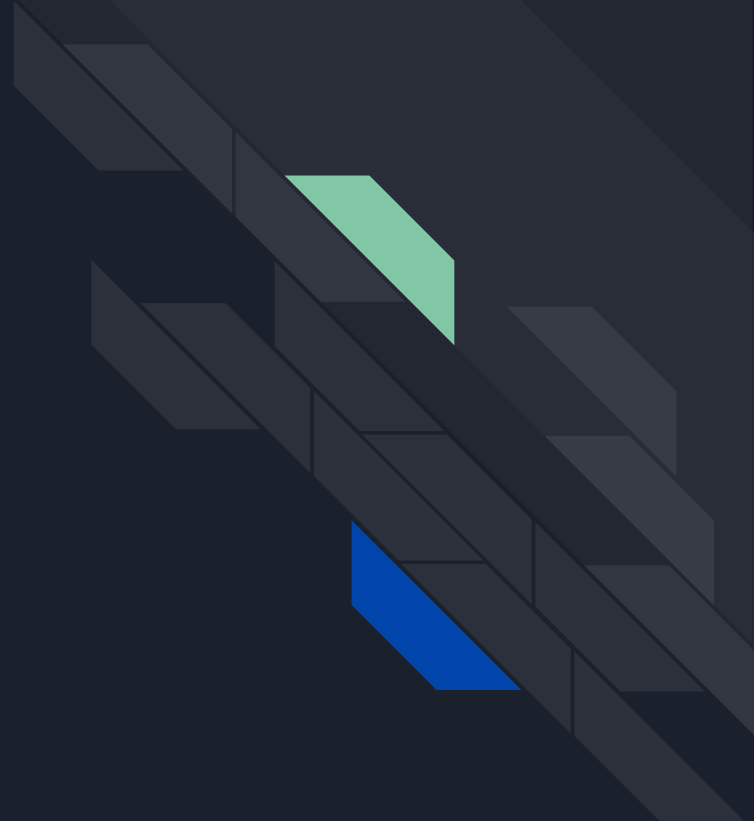


Data Quality

- Clean Dataset - 0 missing, duplicate, negative or invalid values)
- isFlaggedFraud excluded

Part 2

Methodology



Pipeline



TOOLS

Kaggle

Polars

Parquet

LightGBM

MLflow

FastAPI



Feature Engineering

- Balance Engineering
- Time Features
- Log Transforms
- Transaction and Account encoding



Training

Trained using LightGBM

TRAIN 60%

VAL 20%

TEST 20%

1000

boosting rounds

0.05

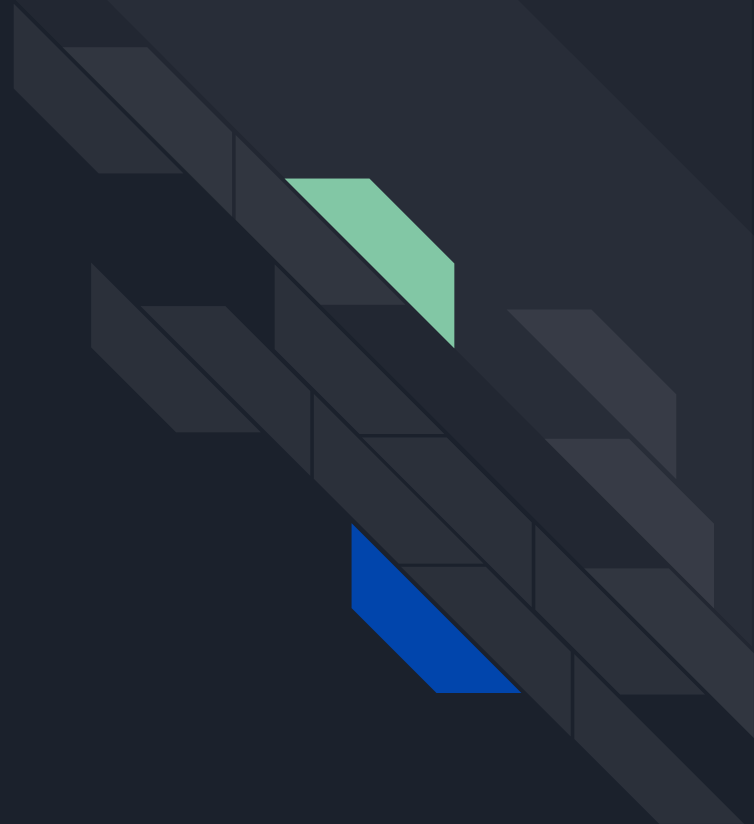
learning rate

127

max leaves

Part 3

Results



Near-perfect fraud detection on PaySim.

LightGBM with balance engineering features, evaluated on held-out test set.

0.9986

PR-AUC

Precision-Recall Area

0.9988

F1 Score

Precision and recall balanced

1,639 / 1,643

Fraud Cases Caught

0 false alarms on 1.27M transactions

Takeaway: balance engineering features drive near-perfect detection — confirmed by ablation.

Balance engineering carries almost everything.

Each group removed once. Drop in PR-AUC vs. baseline (0.9986) shown below.

FEATURE GROUP REMOVED	PR-AUC DROP	INTERPRETATION
Balance Engineering	-0.4967	Model becomes a coin flip
Account Type	-0.0010	Negligible impact
Transaction Type	-0.0003	Negligible impact
Time Features	-0.0001	Negligible impact
Raw Balances (log)	+0.0002	Redundant — engineered features cover it

Removing balance engineering drops PR-AUC by 0.4967 — the model halves in quality. All other groups are replaceable.

\$57.70 returned for every \$1 spent investigating.

Assumptions: avg. fraud amount \$1,467 (PaySim), \$25 per investigation. Evaluated on test set of 1.27M transactions.

\$2.4M

Revenue Saved

1,639 fraud cases caught

\$41K

Investigation Cost

0 false alarms generated

\$2.36M

Net Value

Revenue minus investigation cost

\$5,868

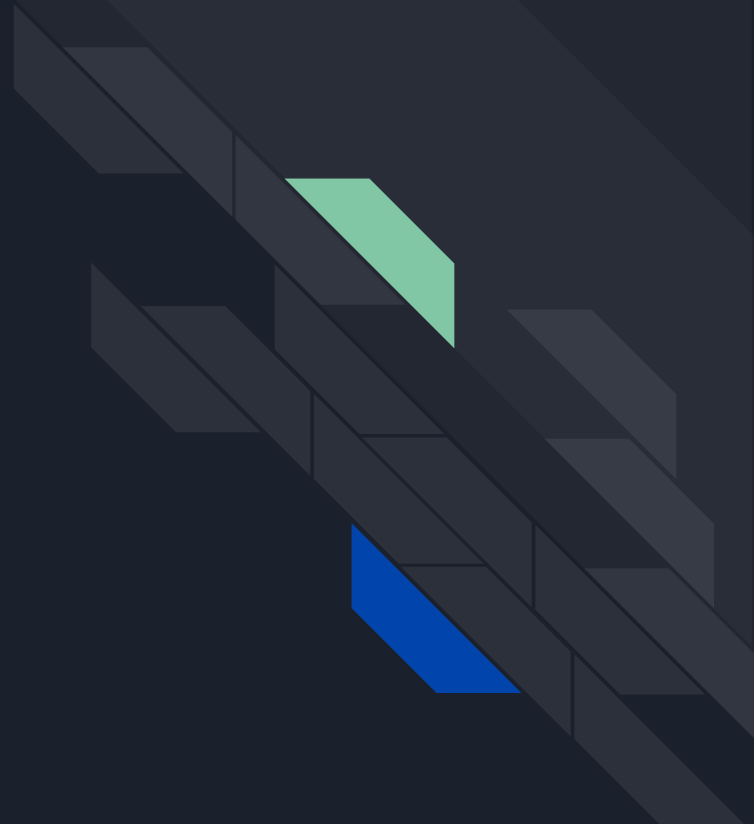
Missed Fraud Cost

Only 4 cases missed

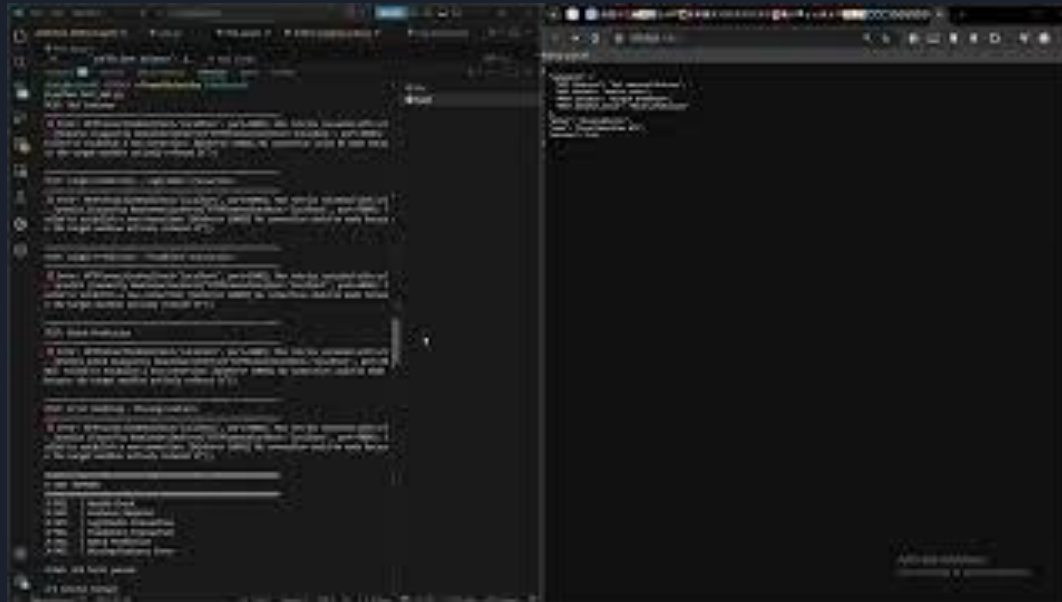
The model catches 99.8% of fraud with zero false alarms — investigation teams never waste time on legitimate transactions.

Part 4

Demo and Wrap-up



Live Demo



Link: <https://www.youtube.com/watch?v=EcGTdas3m4>



Limitations

- Synthetic data
- No temporal validation
- Point forecasts only
- No external data
- No cost-sensitive metrics



Key Takeaways / Future Improvements

1. Handle imbalance aggressively
2. Balance engineering does heavy lifting
3. Engineering around the model matters



Thank You

Questions?

Repository

<https://github.com/Big-Data-Fraud-Detection/Fraud-Detection>